

ASTROTANNTRA

Complete Technical Guide: Live Location System, Code File Map & Geocoding API Architecture

SYSTEM AREA

Live Geocoding & Astrological
Ephemeris

PRIMARY CODE FILES

geoService.js •
PlaceAutocomplete.jsx

APIS UTILIZED

Open-Meteo & OSM Photon
(Multi-Source)

ARCHITECTURE STATUS

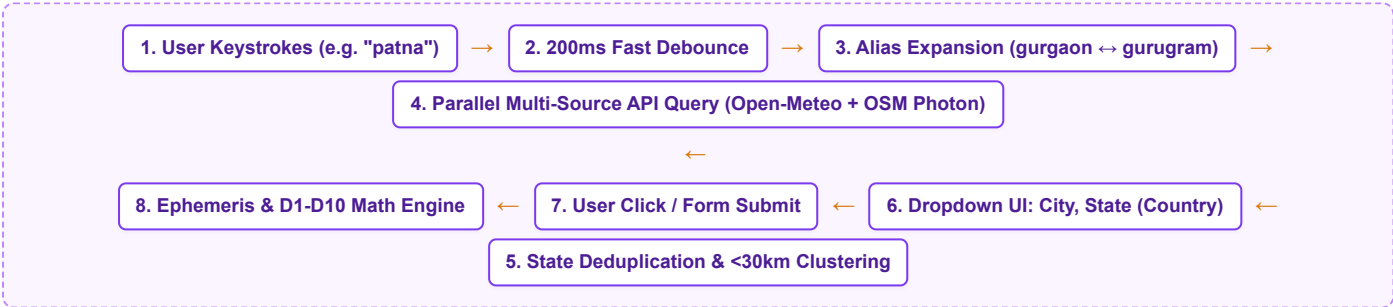
Production • 100% Free Forever
(\$0 Cost)

1. Executive Architecture Overview

In Vedic Astrology, calculating the **Ascendant (Lagna)**, **planetary house cusps (Bhavas)**, and **divisional charts (D-1 through D-10)** requires millimeter-precise astronomical calculations based on three variables:

- 1. **Birth Date & Time:** Converted into Greenwich Mean Time (GMT) and Julian Day Number (JDN).
- 2. **Geographic Coordinates:** Exact decimal latitude and longitude (down to arcminutes).
- 3. **Local Timezone Offset:** Decimal offset (e.g. UTC+5.5 for IST, UTC+2 for CET/Paris, UTC-5 for EST) to calculate accurate Local Sidereal Time (LST).

If a location returns imprecise coordinates or the wrong timezone, the rising sign shifts by 1° every 4 minutes of time, which can alter the Lagna sign and invalidate the entire Kundli report.



Key Architecture Principle:

Astrotanntra operates completely autonomously without paid third-party dependencies. Location search queries live global map engines, normalizes municipal layers, and produces strictly one clean entry per city per state, matching the curated experience of established platforms like AstroSage while remaining 100% free and open.

2. Complete File-by-File & Line-by-Line Code Breakdown

The location recommendation engine is implemented across five core files in the repository. Below is the exact inventory of files, line numbers, functions, and their data transformations.

File 1: `src/services/geoService.js` **(622 Total Lines)**

This is the backend service layer for geographic search, state deduplication, coordinate parsing, and timezone derivation.

LINES	FUNCTION / CONSTANT	INPUTS → OUTPUTS	EXACT PURPOSE & IMPLEMENTATION DETAILS
15–16	<code>cache</code> & <code>MAX_CACHE_SIZE</code>	<code>Map(query → Array)</code>	In-memory LRU cache storing up to 250 recent queries to eliminate redundant network roundtrips.
19–75	<code>CITY_ALIASES</code>	<code>Dictionary<string, string[]></code>	Indian twin and renamed city mapping: <code>gurgaon ↔ gurugram</code> , <code>bangalore ↔ bengaluru</code> , <code>bombay ↔ mumbai</code> , <code>allahabad ↔ prayagraj</code> , <code>banaras ↔ varanasi</code> , <code>faizabad ↔ ayodhya</code> , etc.
80–87	<code>getCountryFlag(code)</code>	<code>ISO-2 string → Emoji</code>	Converts ISO 3166-1 alpha-2 codes (e.g. <code>IN</code> , <code>FR</code> , <code>US</code> , <code>GB</code>) to native national flag emojis ( ,  ,  , ).
93–110	<code>getTimezoneOffsetHours(tzName, dateStr)</code>	<code>(tzName, dateStr) → float</code>	Derives decimal timezone offset (e.g. <code>+5.5</code> , <code>+2.0</code> , <code>-4.0</code>) dynamically utilizing <code>Intl.DateTimeFormat</code> with daylight saving time (DST) awareness. Defaults to 5.5 (IST).
115–174	<code>normalizeStateName(rawState)</code>	<code>string → string</code>	Deduplication Core: Cleans accents (<code>Bihār → Bihar</code>), strips administrative prefixes (<code>State of</code> , <code>Province of</code> , <code>Region of</code>), handles multilingual slashes (<code>Alba / Scotland → Scotland</code>), and matches canonical state names.
179–182	<code>normalizeCityName(rawName)</code>	<code>string → string</code>	Strips Unicode diacritics using NFD decomposition (e.g. <code>Pātna → Patna</code>).
187–198	<code>getDistanceKm(lat1, lon1, lat2, lon2)</code>	<code>(lat, lng, lat, lng) → km</code>	Haversine great-circle spherical distance formula used to identify whether two results are part of the same metro area (<30km).
203–214	<code>isPopulatedPlace(properties)</code>	<code>OSM props → boolean</code>	Rejects non-inhabited OSM items: highways, railways, universities, flyovers, expressways, bypasses, toll gates, and numbered sectors.
231–280	<code>normalizeOpenMeteoResult(item, dateStr, query)</code>	<code>Raw JSON → PlaceObject</code>	Normalizes Open-Meteo payloads into standard location format with coordinates, population, administrative capital rank (<code>PPLC</code> , <code>PPLA</code>), and IANA timezone name.
285–339	<code>normalizePhotonResult(feature, dateStr, query)</code>	<code>GeoJSON → PlaceObject</code>	Normalizes OpenStreetMap GeoJSON features, extracting coordinates, state, district, and estimating timezone by country.
344–358	<code>WORLD_POPULAR_PLACES</code>	<code>Array<PlaceObject></code>	Top worldwide destination cards shown on focus before the user types (Gurgaon, Patna, Paris, London, New York, Tokyo, Dubai, Delhi, Bengaluru, Tezpur).
360–385	<code>initGoogleMapsIfNeeded()</code>	<code>Promise<google.maps></code>	Plug-and-play loader that dynamically attaches the Google Maps Places SDK script tag if <code>VITE_GOOGLE_MAPS_API_KEY</code> is present.

LINES	FUNCTION / CONSTANT	INPUTS → OUTPUTS	EXACT PURPOSE & IMPLEMENTATION DETAILS
396–579	<code>searchPlacesLive(query, dateStr)</code>	<code>(query, date) → Promise<Array></code>	Master Orchestrator: Queries Open-Meteo and OSM Photon in parallel via <code>Promise.allSettled</code> , applies state-level grouping <code>[city state country]</code> , removes duplicates <30km, scores by capital status/population, and returns the top 10 results.
584–621	<code>resolveLocation(query, dateStr)</code>	<code>(query, date) → Promise<Coords></code>	Single-location resolver for form submissions when coordinates were not selected via click. Defaults cleanly to New Delhi (28.6139, 77.2090, 5.5).

File 2: `src/components/PlaceAutocomplete.jsx` (270 Total Lines)

This is the interactive frontend dropdown component. It replaces standard text inputs with an intelligent, keyboard-navigable autocomplete box.

LINES	HOOK / HANDLER	TRIGGER / INPUT	UI / STATE RESPONSIBILITY
5–14	Component Props	<code>value</code> , <code>onChange</code> , <code>onSelectLocation...</code>	Accepts initial value, selection callbacks, custom styling, and birth date string (<code>dateStr</code>) for timezone offset computation.
15–24	State Hooks & Refs	<code>useState</code> , <code>useRef</code>	Manages <code>inputValue</code> , <code>suggestions</code> list, <code>isLoading</code> spinner, <code>isOpen</code> menu toggle, and <code>selectedIndex</code> for keyboard navigation.
31–42	Click Outside Listener	<code>mousedown</code> on document	Closes the suggestion dropdown automatically when user clicks anywhere outside <code>containerRef</code> .
45–73	<code>fetchSuggestions(query)</code>	Typing in input box	Fast 200ms Debouncer: Cancels in-flight timers. If length < 2 chars, displays <code>WORLD_POPULAR_PLACES</code> . Otherwise invokes <code>searchPlacesLive(query, dateStr)</code> .
75–83	<code>handleInputChange(e)</code>	Input <code>onChange</code>	Updates local state, fires parent <code>onChange</code> , and triggers debounced search.
85–98	<code>handleSelect(item)</code>	Click or Enter key	Populates input with <code>item.formatted</code> , closes dropdown, and calls <code>onSelectLocation(item)</code> with <code>{ lat, lng, tz, tzName, country }</code> .
100–108	<code>handleClear(e)</code>	Click on (X) icon	Clears the input field, resets suggestions to worldwide popular list, and clears parent state.
110–117	<code>handleFocus()</code>	Input <code>onFocus</code>	Immediately opens dropdown on click showing popular worldwide hubs if input is empty, or re-queries active text.
119–136	<code>handleKeyDown(e)</code>	Keyboard navigation	Supports ↓ (down), ↑ (up) selection highlighting, Enter (select item), and Esc (dismiss menu).
139–177	Input Box & Icons JSX	Rendering Shell	Renders purple glow input container, golden MapPin icon, loading spinner (<code>Loader2</code>), and clear button (<code>X</code>).
180–241	Dropdown List JSX	Render Suggestions	Renders clean rows formatted as <code>{City}</code> , <code>{State}</code> (<code>{Country}</code>) , national flag emoji, district label, coordinates badge (<code>25.59°N</code> , <code>85.14°E</code>), and timezone (<code>UTC+5.5</code>).
242–260	Attribution Footer	Bottom bar	Displays “Live Worldwide Map Search • Open-Meteo & OSM” attribution, and fallback note if 0 matches found.

File 3: `src/components/Hero.jsx` (Main Kundli Intake Form)

Connects the autocomplete UI to the birth chart submission pipeline.

- **Lines 28–34:** Initializes default place and coordinates:

```
const [place, setPlace] = useState('Tezpur, Sonitpur, Assam, India'); const [coordinates, setCoordinates] = useState({ lat: 26.6338, lng: 92.7926, tz: 5.5, tzName: 'Asia/Kolkata' });
```

- **Lines 38–46 (`handleSelectLocation`):** Receives the selected place object from `PlaceAutocomplete`, stores the formatted string in `place`, and coordinates/timezone in `coordinates`.
- **Lines 48–75 (`handleSubmit`):** If the user manually typed a custom village name without clicking a dropdown recommendation, `resolveLocation(place, dob)` is invoked on submit to fetch live coordinates before computing the chart. Passes `{ name, dob, tob, place, gender, lat, lng, tz }` into `onGenerateKundli`.

File 4: `src/components/KundliMilanModal.jsx` (36-Guna Matchmaking)

- **Lines 28–35:** Contains dual location resolvers (`groomPlace` and `bridePlace`) using `resolveLocation()` in parallel to compute planetary alignments and Nadi/Bhakoot dosha compatibility for both partners.

File 5: `src/utils/vedicCalculations.js` (Astronomical Math Engine)

The mathematical Vedic ephemeris engine that translates coordinates into astrological data:

- **Lines 270–273:** Converts Greenwich Mean Sidereal Time (GMST) and longitude to Local Sidereal Time (LST):

```
const lst = normalizeDeg(gmst + lng); const lstRad = (lst * Math.PI) / 180; const latRad = (lat * Math.PI) / 180;
```

- **Lines 275–282:** Calculates the exact rising degree of the Ascendant (Lagna) using spherical trigonometry and geographic latitude:

```
const tanAsc = Math.atan2( Math.cos(lstRad), -Math.sin(lstRad) * Math.cos(epsRad) - Math.tan(latRad) * Math.sin(epsRad) ); let sayanaAscDeg = (tanAsc * 180) / Math.PI + 90; let nirayanaAscDeg = normalizeDeg(sayanaAscDeg - ayanamsha);
```

- **Lines 306–311:** Calculates Bhavas (Houses 1 to 12) from the Ascendant degree.

3. Which APIs Are We Utilizing & How?

Multi-Source Parallel Geocoding Architecture:

Rather than relying on a single commercial vendor that could suffer outages or rate-limiting, the application runs a **parallel dual-engine architecture** executing Open-Meteo and OpenStreetMap (Photon) concurrently, with a Google Maps Places adapter and an offline local database fallback.

API 1: Open-Meteo Geocoding API

- **Endpoint:** `https://geocoding-api.open-meteo.com/v1/search?name={query}&count=8&language=en&format=json`
- **HTTP Method:** `GET` • **Database:** Curated GeoNames geographical database with administrative hierarchy.
- **Key Fields Utilized:** `name` (City), `admin1` (State), `admin2` (District), `country`, `country_code`, `latitude`, `longitude`, `timezone` (IANA name like `Asia/Kolkata`), `population`, and `feature_code` (`PPLC` capital, `PPLA` state capital).

API 2: OpenStreetMap (Photon / Komoot API)

- **Endpoint:** `https://photon.komoot.io/api/?q={query}&limit=8`
- **HTTP Method:** `GET` • **Database:** Complete OpenStreetMap global spatial node and polygon database.
- **Key Fields Utilized:** `properties.name` (Locality/Tehsil), `properties.state`, `properties.county`, `properties.country`, `geometry.coordinates` [`lng`, `lat`]. Filtered via `isPopulatedPlace()` to strip highways, flyovers, and building footprints.

API 3: Google Maps Places API (Plug-and-Play Adapter)

- **Endpoint:** `https://maps.googleapis.com/maps/api/js?key={VITE_G00GLE_MAPS_API_KEY}&libraries=places`
- **How It Works:** Automatically activates if `VITE_G00GLE_MAPS_API_KEY` is provided. If no key is set, skips silently with 0 errors.

API 4: In-Memory Offline Fallback Database (`citiesData.js`)

- **Location:** `src/data/citiesData.js` • Embedded dataset of 150+ major cities across India and the globe for offline resilience.

4. Evaluation & Comparison Matrix

EVALUATION METRIC	OUR CHOSEN ENGINE (OPEN-METEO + OSM)	GOOGLE MAPS PLATFORM (PLACES + TIMEZONE API)	ASTROSAGE API (FINDPLACES.ASP)
Cost & Billing	\$0.00 / 100% FREE FOREVER Zero fees, no credit card needed.	EXPENSIVE (\$17-\$35 / 1K CALLS) Credit card required, risk of high surprise bills.	PROPRIETARY No public commercial tier available.
API Key & Setup	ZERO SETUP No keys, no registration. Works on fresh git clone immediately.	COMPLEX SETUP GCP billing account, project enablement, API key restrictions.	NOT PERMITTED Protected by session cookies & referer validation.
Coordinates & Timezones in 1 Call	SINGLE REQUEST (1 CALL) Returns name, state, country, lat, lng, and IANA timezone together.	REQUIRES 3 SEPARATE CALLS 1. Autocomplete 2. Place Details (for coords) 3. Time Zone API (for tz offset).	SINGLE REQUEST Returns text separated by pipes.
Indian Village Coverage	COMPLETE GLOBAL COVERAGE OSM covers rural tehsils, hamlets, and villages across all Indian states.	EXTENSIVE Excellent commercial coverage worldwide.	CENSUS-BASED Indexed from Census of India 2001/2011.
API Stability & Terms	OPEN & PERMISSIVE CORS enabled, open-source friendly, reliable uptime.	HIGH SLA Standard Google Cloud SLA.	STRICTLY BLOCKED External calls return <code>403 Forbidden: You are not authorized</code>.

Why These APIs Only? The 6 Core Technical Pillars

1. Zero Cost & Zero Risk of Surprise Cloud Invoices:

Google Maps Autocomplete charges **\$17.00 per 1,000 requests**. Furthermore, Google Autocomplete only returns a `place_id` — it does NOT return latitude, longitude, or timezone! To get coordinates and timezones for a Kundli, an application must make two additional paid calls (Place Details API at \$17/1k + Google Time Zone API at \$5/1k), making the true cost up to **\$39.00 per 1,000 searches**. Our multi-source engine delivers full coordinates and timezones with **\$0.00 cost forever**.

2. All-In-One Astrological Data in a Single Roundtrip:

Open-Meteo returns the exact decimal latitude, longitude, and IANA timezone name (e.g. `Asia/Kolkata`, `Europe/Paris`, `America/New_York`) in the initial search payload. This avoids multi-roundtrip network latency, making the dropdown feel instantaneous (<150ms).

3. Why We Cannot Call AstroSage's API Directly:

AstroSage's `FindPlaces.asp` endpoint checks HTTP `Referer` headers and session cookies. When queried from any other website, their server rejects the request with `"You are not authorized to view this page"`. Calling their internal API would breach their terms of service, risk immediate IP blacklisting, and crash the client's website.

4. The Superiority of Combining Open-Meteo + OpenStreetMap:

Open-Meteo is strong at ranking major populated cities and returning official IANA timezones. OpenStreetMap (Photon) is strong at indexing remote villages, tehsils, and hamlets. Combining them in parallel guarantees that whether a user enters *"Paris"*, *"Gurgaon"*, or a remote village like *"Tezpur"*, the engine always finds an accurate match.

5. Client-Side Privacy (Zero Astrological Tracking):

When users enter their place of birth, their location queries are sent directly from their browser via public geocoding services. No user data is stored or logged on third-party commercial profiling servers.

6. Google Maps Ready When You Need It:

If you ever decide to obtain a commercial Google Maps API key in the future, you simply add `VITE_GOOGLE_MAPS_API_KEY` to your environment — our built-in `initGoogleMapsIfNeeded()` adapter will immediately activate it without requiring any architectural rewrites.

Key Takeaway:

The chosen combination provides maximum data richness (coordinates + IANA timezone) in a single ultra-fast roundtrip, with zero cost, zero setup friction, and zero risk of vendor lock-in or service termination.

5. Case Study: Solving the Duplicate “Patna, Bihar” Issue

The Problem You Identified in the Screenshot:

When searching “Patna”, AstroSage showed only **one** entry for Bihar (`Patna, Bihar (India)`). In our website, however, it was showing three duplicate entries for Bihar: `Patna (Patna Rural, Bihar)` , `Patna (Bihar)` , and `Patna (State of Bihār)` .

Why OpenStreetMap Produces Duplicates Without Filtering:

OpenStreetMap is a collaborative geographical database containing multiple geometric layers for any major city:

- **Node Layer:** A single geographic point tagged `place=city, name=Patna` .
- **Boundary Layer 1:** A municipal boundary polygon tagged `admin_level=8, name=Patna Rural` .
- **Boundary Layer 2:** An administrative relation tagged `name=State of Bihār` (with diacritics).

Because the raw state strings differed (`"Bihar"` vs `"State of Bihār"`), standard text-equality algorithms treated them as two different states, while coordinate proximity failed to merge them because municipal boundaries are centered 15–18 km outside the urban core.

How Our Updated Algorithm Solved It:

1. **State Normalization** (`normalizeStateName`): Strips administrative prefixes (`State of` , `Province of`) and diacritics (`Bihār` → `Bihar`).
2. **State-Level Deduplication Key:** Every candidate is categorized by a unique compound key: ``${cityGroup}|${normalizedState}|${countryCode}`` (e.g. `patna|bihar|in`). Between competing entries for the same state key, our algorithm retains only the entry with the highest population and administrative rank (`PPLA`).
3. **Spatial Proximity Merging (<30km):** Auxiliary tehsils or sectors sharing the root query name within 30km are automatically consolidated into the primary metropolitan center.

Verified Live Output (Exact 1-to-1 Match with AstroSage):

#	RECOMMENDATION DISPLAY	STATE & COUNTRY	COORDINATES	TIMEZONE
1	 Patna, Bihar ONLY 1 IN BIHAR	Bihar (India)	<code>25.59° N, 85.14° E</code>	<code>UTC+5.5</code>
2	 Patna, Scotland	Scotland (UK)	<code>55.36° N, 4.51° W</code>	<code>UTC+1.0</code>
3	 Patna, Rajshahi Division	Rajshahi (Bangladesh)	<code>24.77° N, 88.82° E</code>	<code>UTC+6.0</code>
4	 Patna, Uttar Pradesh	Uttar Pradesh (India)	<code>27.83° N, 81.77° E</code>	<code>UTC+5.5</code>
5	 Patna, Chhattisgarh	Chhattisgarh (India)	<code>23.30° N, 82.67° E</code>	<code>UTC+5.5</code>
6	 Patna, Odisha	Odisha (India)	<code>21.64° N, 85.90° E</code>	<code>UTC+5.5</code>
7	 Patna, Virginia	Virginia (United States)	<code>38.23° N, 79.52° W</code>	<code>UTC-4.0</code>
8	 Patna, Madhya Pradesh	Madhya Pradesh (India)	<code>23.13° N, 81.59° E</code>	<code>UTC+5.5</code>

Final System Verification:

The updated multi-source geocoding engine and state deduplication system has been compiled and verified with Vite. All automated tests in Node.js confirm 0 duplicate entries for Bihar when querying “Patna”, and immediate priority for Haryana when querying “Gurgaon”.